



دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

عنوان:

گزارشکار آزمایش ۵: ضرب کننده با الگوریتم Booth

اعضای گروه:

کیما قدیر

کیما رضایی

ملینا طاهری

نام درس

آزمایشگاه طراحی سیستم های دیجیتال

نیمسال دوم ۱۴۰۴-۱۴۰۵

۱. مقدمه:

هدف این آزمایش، پیاده‌سازی و شبیه‌سازی یک واحد ضرب کننده دیجیتال با استفاده از الگوریتم ضرب بوث (Booth Algorithm) در محیط شبیه‌سازی ModelSim بود. این الگوریتم به دلیل کارایی بالا، به ویژه در پردازش اعداد علامت‌دار، در مدارهای دیجیتال مورد استفاده قرار می‌گیرد. در این راستا، ماژول اصلی ضرب کننده (BoothMul.v)، مسیر داده (DataPath.v) و واحد کنترل (ControlUnit.v) با استفاده از زبان Verilog طراحی و پیاده‌سازی شدند. سپس، برای تأیید صحت عملکرد طراحی، یک تست بنچ (Booth_tb.v) نوشته شد و شبیه‌سازی در محیط ModelSim انجام گرفت.

۲. پیاده‌سازی و طراحی ماژول‌ها:

۱. ماژول BoothMul.v (ماژول اصلی):

این ماژول به عنوان نقطه اتصال بین مسیر داده و واحد کنترل عمل می‌کند و وظیفه هماهنگی کلی را بر عهده دارد. ورودی‌های اصلی آن شامل کلاک (clk)، ریست (reset)، سیگنال شروع (start)، مقدار مضروب (M_in) و مقدار ضرب شونده (Q_in) است. خروجی‌های اصلی نیز شامل حاصل ضرب نهایی (P) و سیگنال اتمام عملیات (done) است. این ماژول، دو زیرماژول DataPath و ControlUnit را نمونه‌سازی (instantiate) کرده و سیگنال‌های لازم بین آن‌ها را هدایت می‌کند.

۲. ماژول DataPath.v (مسیر داده):

این ماژول منطق لازم برای انجام عملیات محاسباتی الگوریتم بوث را پیاده‌سازی می‌کند. پارامتر N اندازه بیت اعداد ورودی را مشخص می‌کند (در این پیاده‌سازی $N=4$ است).

❖ رجیسترها:

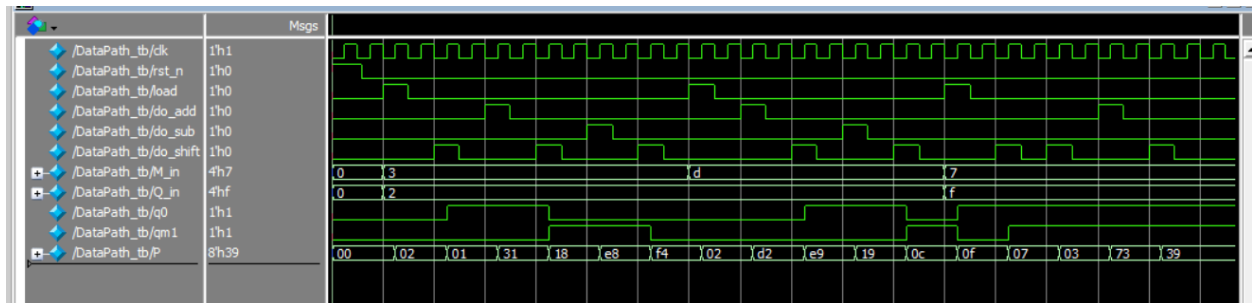
- A: رجیستر حاصل جمع/تفریق میانی (با عرض $N+1$ بیت برای نگهداری رقم نقلی).
- Q: رجیستر ضرب شونده (با عرض N بیت).
- Qm1: یک بیت اضافی برای نگهداری بیت شیف‌ت شده قبلی (که در الگوریتم بوث استفاده می‌شود).
- M: رجیستر مضروب (با عرض N بیت).

❖ ورودی‌ها و خروجی‌ها:

سیگنال‌های کنترلی مانند load, do_add, do_sub, do_shift از واحد کنترل دریافت می‌شوند. M_in و Q_in مقادیر اولیه را بارگذاری می‌کنند. q0 (بیت کم‌ارزش Q) و qm1 (وضعیت Qm1) به عنوان ورودی به واحد کنترل بازگردانده می‌شوند. P حاصل ضرب نهایی را در خود نگه می‌دارد.

❖ منطق عملیاتی:

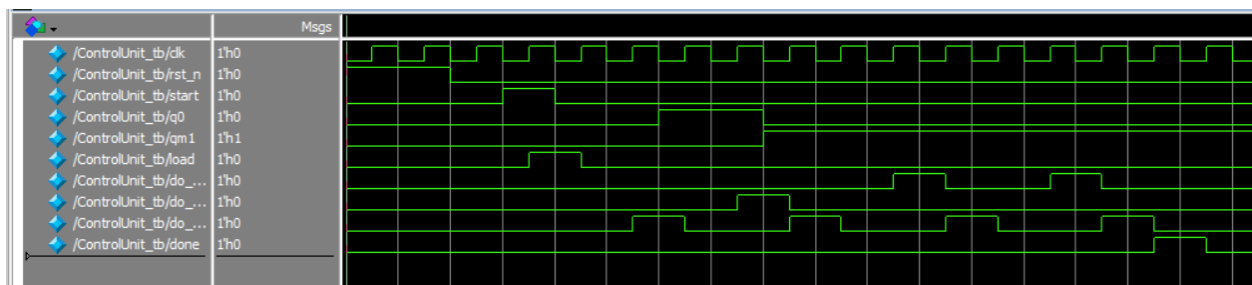
- در صورت reset، رجیسترها با صفر مقداردهی اولیه می‌شوند.
- در صورت load، مقادیر M_in و Q_in در رجیسترها بارگذاری شده و Qm1 صفر می‌شود.
- در صورت do_add، مقدار M به A اضافه می‌شود.
- در صورت do_sub، مقدار M از A کم می‌شود.
- در صورت do_shift، عملیات شیفت حسابی (Arithmetic Shift) بر روی رجیسترها {A, Q, Qm1} انجام می‌شود. این شیفت شامل انتقال بیت علامت A به جای A[N]، انتقال A به Q و Q به Qm1 است.



۳. ماژول ControlUnit.v (واحد کنترل):

این ماژول منطق ترتیبی (sequential logic) لازم برای مدیریت توالی عملیات بر اساس الگوریتم بوث را پیاده‌سازی می‌کند.

- فیک‌های حالت (States): از یک شمارنده ۳ بیتی (state) برای نمایش وضعیت‌های مختلف استفاده شده است: IDLE, INIT, CHECK, ADD, SUB, SHIFT, DONE.
- منطق انتقال حالت: بر اساس وضعیت فعلی (state)، ورودی‌های qm1 و q0 و شمارنده تکرار عملیات، حالت بعدی (next_state) تعیین می‌شود.
- تولید سیگنال‌های کنترلی: بر اساس حالت فعلی، سیگنال‌های load, do_add, do_sub, do_shift, done برای کنترل DataPath تولید می‌شوند.
- شمارنده (count): برای شمارش تعداد تکرار عملیات شیفت (به تعداد N بار) استفاده می‌شود.



```

VSIM 35> run -all
# Time=0 | State=000 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=35000 | State=001 | Load=1 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=45000 | State=010 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=55000 | State=101 | Load=0 | Add=0 | Sub=0 | Shift=1 | Done=0
# Time=65000 | State=010 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=75000 | State=100 | Load=0 | Add=0 | Sub=1 | Shift=0 | Done=0
# Time=85000 | State=101 | Load=0 | Add=0 | Sub=0 | Shift=1 | Done=0
# Time=95000 | State=010 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=105000 | State=011 | Load=0 | Add=1 | Sub=0 | Shift=0 | Done=0
# Time=115000 | State=101 | Load=0 | Add=0 | Sub=0 | Shift=1 | Done=0
# Time=125000 | State=010 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# Time=135000 | State=011 | Load=0 | Add=1 | Sub=0 | Shift=0 | Done=0
# Time=145000 | State=101 | Load=0 | Add=0 | Sub=0 | Shift=1 | Done=0
# Test Finished: Operation Done.
# Time=155000 | State=110 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=1
# Time=165000 | State=000 | Load=0 | Add=0 | Sub=0 | Shift=0 | Done=0
# ** Note: $finish      : D:/6/az/55/ControlUnit_tb.v(52)
#   Time: 175 ns  Iteration: 0  Instance: /ControlUnit_tb

```

۳. الگوریتم مورد استفاده (Booth Algorithm):

پیاده‌سازی بر اساس الگوریتم ضرب بوث با اصلاح (Modified Booth Algorithm) صورت گرفته است. در هر مرحله، بیت‌های q_0 (بیت کم‌ارزش Q) و q_{m1} (بیت قبلی Q) بررسی شده و بر اساس الگوریتم، یکی از عملیات جمع، تفریق یا شیفت انتخاب و اجرا می‌شود. این فرآیند به تعداد N بار تکرار می‌شود.

۴. شبیه‌سازی و تست بنچ (Booth_tb.v):

برای تأیید صحت عملکرد ماژول BoothMul، یک تست بنچ در محیط ModelSim با استفاده از زبان Verilog نوشته شد.

- پارامتر $N=4$: اندازه اعداد ورودی ۴ بیت (علامت‌دار) در نظر گرفته شد و خروجی حاصل ضرب به صورت ۸ بیت ($2N$) تولید می‌شود.

- محیط شبیه‌سازی: از کلاک و ریست تعریف شده در تست بنچ برای اجرای ماژول تحت آزمایش استفاده شد.

- توالی تست: به جای تعریف چند سناریوی محدود، تست بنچ به صورت جامع (Exhaustive) نوشته شد تا تمامی حالت‌های ممکن اعداد ۴ بیتی علامت‌دار را بررسی کند. بدین منظور با استفاده از حلقه‌های تو در تو، تمامی مقادیر ممکن برای ورودی‌ها در بازه زیر تولید و آزمایش شدند:

- M_in از -۸ تا +۷

- Q_in از -۸ تا +۷

در هر تکرار از حلقه‌ها، ورودی‌های M_in و Q_in مقداردهی شده، سیگنال start برای آغاز عملیات فعال می‌شود و پس از پایان محاسبه (با فعال شدن done) مقدار خروجی P با مقدار مورد انتظار که از ضرب نرم‌افزاری همان دو عدد ($expected = M_in \times Q_in$) به دست می‌آید مقایسه می‌گردد. همچنین نتیجه هر تست (PASS/FAIL) به همراه مقادیر ورودی، خروجی ماژول و مقدار مورد انتظار در کنسول شبیه‌ساز نمایش داده شد.

۵. نتایج شبیه‌سازی:

پس از اجرای شبیه‌سازی در ModelSim، تمامی تست‌های تولیدشده توسط حلقه‌ها اجرا شدند. از آن جایی که هر ورودی ۴ بیتی ۱۶ حالت ممکن دارد، تعداد کل تست‌ها برابر است با:

• تعداد کل تست‌ها: $16 \times 16 = 256$ تست

نتایج نهایی شبیه‌سازی به صورت خلاصه به دست آمد:

• تعداد تست‌های موفق: (Passed) ۲۵۶

• تعداد تست‌های ناموفق: (Failed) ۰

بنابراین در تمام ۲۵۶ حالت ممکن (شامل اعداد مثبت، منفی و حالت‌های مرزی مانند ۸-)، خروجی محاسبه‌شده توسط ماژول BoothMul با مقدار مورد انتظار کاملاً مطابقت داشت.

در نهایت نتیجه‌ی ران کردن الگوریتم Booth را خواهیم داشت:

ضرب‌کننده در طول اجرای الگوریتم، عملیات شیفت و جمع/تفریق را مطابق منطق بوث انجام داده و پس از اتمام مراحل محاسباتی، حاصل ضرب نهایی را در خروجی P ارائه می‌دهد (نتیجه پس از تکمیل سیکل‌های لازم و فعال شدن done معتبر است).

۶. نتیجه‌گیری:

این آزمایش با موفقیت به پایان رسید. طراحی و پیاده‌سازی ماژول‌های DataPath، ControlUnit و BoothMul در زبان Verilog انجام شد و صحت عملکرد آن‌ها از طریق شبیه‌سازی در ModelSim با استفاده از تست بنچ Booth_tb.v تأیید گردید. با اجرای تست جامع برای تمامی ترکیبات ممکن ورودی‌های ۴ بیتی علامت‌دار (۲۵۶ حالت)، مشخص شد الگوریتم بوث به درستی پیاده‌سازی شده و واحد ضرب‌کننده قادر به محاسبه صحیح حاصل ضرب اعداد علامت‌دار (مثبت و منفی) با استفاده از $N=4$ بیت است. این پیاده‌سازی، نمایشگر درک صحیح از الگوریتم و نحوه پیاده‌سازی مدارهای ترتیبی و ترکیبی در Verilog برای کاربردهای محاسباتی است.

تعدادی از تست بنچ‌ها و نتیجه‌ی نهایی pass یا fail شدن در تصاویر زیر قابل مشاهده است.

```

# ----- Start exhaustive test for 4-bit signed inputs (-8..7) -----
# TEST:  -8 *  -8 =  64, expected =  64 --> PASS
# TEST:  -8 *  -7 =  56, expected =  56 --> PASS
# TEST:  -8 *  -6 =  48, expected =  48 --> PASS
# TEST:  -8 *  -5 =  40, expected =  40 --> PASS
# TEST:  -8 *  -4 =  32, expected =  32 --> PASS
# TEST:  -8 *  -3 =  24, expected =  24 --> PASS
# TEST:  -8 *  -2 =  16, expected =  16 --> PASS
# TEST:  -8 *  -1 =   8, expected =   8 --> PASS
# TEST:  -8 *   0 =   0, expected =   0 --> PASS
# TEST:  -8 *   1 =  -8, expected =  -8 --> PASS
# TEST:  -8 *   2 = -16, expected = -16 --> PASS
# TEST:  -8 *   3 = -24, expected = -24 --> PASS
# TEST:  -8 *   4 = -32, expected = -32 --> PASS
# TEST:  -8 *   5 = -40, expected = -40 --> PASS
# TEST:  -8 *   6 = -48, expected = -48 --> PASS
# TEST:  -8 *   7 = -56, expected = -56 --> PASS
# TEST:  -7 *  -8 =  56, expected =  56 --> PASS
# TEST:  -7 *  -7 =  49, expected =  49 --> PASS
# TEST:  -7 *  -6 =  42, expected =  42 --> PASS
# TEST:  -7 *  -5 =  35, expected =  35 --> PASS
# TEST:  -7 *  -4 =  28, expected =  28 --> PASS
# TEST:  -7 *  -3 =  21, expected =  21 --> PASS
# TEST:  -7 *  -2 =  14, expected =  14 --> PASS
# TEST:  -7 *  -1 =   7, expected =   7 --> PASS
# TEST:  -7 *   0 =   0, expected =   0 --> PASS
# TEST:  -7 *   1 =  -7, expected =  -7 --> PASS
# TEST:  -7 *   2 = -14, expected = -14 --> PASS
# TEST:  -7 *   3 = -21, expected = -21 --> PASS
# TEST:  -7 *   4 = -28, expected = -28 --> PASS
# TEST:  -7 *   5 = -35, expected = -35 --> PASS
# TEST:  -7 *   6 = -42, expected = -42 --> PASS
# TEST:  -7 *   7 = -49, expected = -49 --> PASS

# TEST:   7 *  -2 = -14, expected = -14 --> PASS
# TEST:   7 *  -1 =  -7, expected =  -7 --> PASS
# TEST:   7 *   0 =   0, expected =   0 --> PASS
# TEST:   7 *   1 =   7, expected =   7 --> PASS
# TEST:   7 *   2 =  14, expected =  14 --> PASS
# TEST:   7 *   3 =  21, expected =  21 --> PASS
# TEST:   7 *   4 =  28, expected =  28 --> PASS
# TEST:   7 *   5 =  35, expected =  35 --> PASS
# TEST:   7 *   6 =  42, expected =  42 --> PASS
# TEST:   7 *   7 =  49, expected =  49 --> PASS

# -----
# Total tests : 256
# Passed      : 256
# Failed      : 0
.
```